In an infrastructure or platform operator role, you will find the VM-level signals useful, be that for troubleshooting or planning. Developers may not always have access to the sources in this space, with JVM a potential exception. With the basics out of the way, let's first turn our attention to containerized environments.

3.2.2 Containers

Containers are convenient abstractions that provide application-level dependency management. That is, containers leverage Linux kernel features, including namespaces for isolation (e.g., processes, network stacks, or file system mount points) and cgroups for resource usage management. Containers wrap up these kernel features in a simple and usable API, allowing you to launch applications with certain isolation guarantees. They are well suited to package all the dependencies your app has (from programming language libraries to operating-system-level resources to environment variables) into a single artifact, called a *container image*, and run it on a host. For the container host, the general observations for VMs or bare-metal instances apply—that is, usually depending on journald for events, with `journalctl` as its primary interface.

Specifically, for the containers, signals can come from the container runtime, such as Docker or containerd itself. On the other hand, signals come from the running containers—that is, application-level signals.

For logs, different container runtimes offer different levels of support:

- The Docker daemon (https://docs.docker.com/config/daemon/#read-the-logs) features a full-blown solution, depending on the host environment, typically making the logs available via `journalctl` or in a certain file system location, such as under `/var/log/`.
- With containerd, the assumption is that the environment in which it is used (e.g., Kubernetes) takes care of the log management. In other words, it doesn't support its own end-to-end solution; however, if you run containerd as a systemd unit, journald will capture the output, providing you with some basic information.
- The Kubernetes-specific container runtime interface (CRI; http://mng.bz/0KAJ) is platform agnostic and does not directly support logging. Instead, the CRI defers the concrete log support to the container runtime you use, such as containerd (see previous).

Concerning container logs from the application level, we have to step back a bit. Twelve-factor apps consider treating logs (https://12factor.net/logs) as event streams, encourage the apps to log to `stdout`, and don't concern themselves with log routing. This "log-to-stdout" stance is what you find nowadays with containers as well; Docker has full support for container logs (http://mng.bz/Ke8X), enabling you to manage log input and output via log drivers, such as journald, syslog, and Fluentd, as well as cloud provider–specific formats. For example, to send logs from a Docker container directly to Amazon CloudWatch, you would configure the log driver like so: